

数字逻辑设计

曲龙跃
信息科学与技术学院
qulongyue@hit.edu.cn

逻辑

- “事物发展有其内在的**逻辑**”
- “这个人为人处世，有他自己的**逻辑**”
- “说话、写文章都要讲**逻辑**”
- “按照对方辩友的**逻辑**，岂不是说 ……”
- 帝国主义的**逻辑**和人民的**逻辑**是这样的不同。捣乱，失败，再捣乱，再失败，直至灭亡——这就是帝国主义和世界上一切反动派对待人民事业的**逻辑**，…斗争，失败，再斗争，再失败，再斗争，直至胜利——这就是人民的**逻辑**。

（毛泽东选集第四卷——丢掉幻想，准备斗争）

逻辑学

- 我们思考问题，分析问题，要从**逻辑思维**的高度进行，要独立思考判断，不能只是停留在感官认知层面以及**习惯性**接收和反应客观信息上。要跳出本能的习惯性反应，发挥大脑的逻辑思考能力，去思辨。这要求我们学好逻辑学。
- <https://baike.baidu.com/item/逻辑学/85559>

布尔代数

- 1854年，George Boole发表了《**思维规律**》，布尔代数问世了，数学史上树起了一座新的里程碑。
- 布尔代数是一个**集合A**，其上定义了以下结构：
 - 二元运算 $\wedge: A \times A \rightarrow A$ （与、合取）
 - 二元运算 $\vee: A \times A \rightarrow A$ （或、析取）
 - 一元运算 $\neg: A \rightarrow A$ （非、否定）
 - 零元运算（常数）0和1。



George Boole

布尔代数的特点

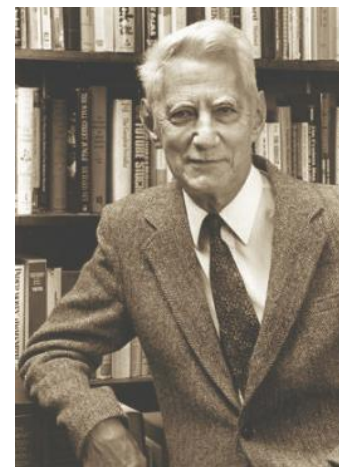
- 这些运算满足以下条件： $\forall a, b, c \in A$

$a \vee (b \vee c) = (a \vee b) \vee c$	$a \wedge (b \wedge c) = (a \wedge b) \wedge c$	结合律
$a \vee b = b \vee a$	$a \wedge b = b \wedge a$	交换律
$a \vee (a \wedge b) = a$	$a \wedge (a \vee b) = a$	吸收律
$a \vee (b \wedge c) = (a \vee b) \wedge (a \vee c)$	$a \wedge (b \vee c) = (a \wedge b) \vee (a \wedge c)$	分配律
$a \vee \neg a = 1$	$a \wedge \neg a = 0$	互补律

- 一般要求布尔集**至少有两个**不同的元素0和1，而且其元素对三种运算 $\wedge$ ， $\vee$ ， $'$ 都**封闭**

布尔代数——>开关代数/逻辑代数

- 布尔代数不仅可以在数学领域内实现集合运算，更广泛应用于电子学、计算机硬件、计算机软件等领域的逻辑运算：当集合内只包含两个元素1和0时，分别对应{真}和{假}，可以用于实现对逻辑的判断。
- 1938年，就读于MIT的香农发表了他那篇著名的硕士论文《继电器与开关电路的符号分析》，奠定了数字电路的理论基础。



Claude E. Shannon

原文表述

《继电器与开关电路的符号分析》

- In the control and protective circuits of complex electrical systems it is frequently necessary to make intricate interconnections of relay contacts and switches. **Examples** of these circuits occur in automatic telephone exchanges, industrial motor-control equipment, and in almost any circuits designed to perform complex operations automatically. In this paper a **mathematical analysis** of certain of the properties of such networks will be made. Particular attention will be given to the problem of **network synthesis**. Given certain characteristics, it is required to find a circuit incorporating these characteristics. The solution of this type of problem is not unique and methods of finding those particular circuits requiring the least number of relay contacts and switch blades will be studied. Methods will also be described for **finding any number of circuits equivalent to a given circuit** in all operating characteristics. It will be shown that several of the well-known theorems on impedance networks have roughly analogous theorems in relay circuits. Notable among these are the delta-wye and star-mesh transformations, and the duality theorem.

原文表述

《继电器与开关电路的符号分析》

- The method of attack on these problems may be described briefly as follows: **any circuit is represented by a set of equations**(the terms of the equations corresponding to the various relays and switches in the circuit). A calculus is developed for manipulating these equations by simple mathematical processes, most of which **are similar to ordinary algebraic algorisms**. This calculus is shown to be exactly **analogous to the calculus of propositions used in the symbolic study of logic**. For the synthesis problem the desired characteristics are first written as a system of equations, and **the equations are then manipulated into the form representing the simplest circuit**. The circuit may then be immediately drawn from the equations. By this method it is always possible to find the simplest circuit containing only series and parallel connections, and in some cases the simplest circuit containing any type of connection.
- Our notation is taken chiefly from symbolic logic. Of the many systems in common use we have chosen the one which seems simplest and most suggestive for our interpretation. Some of our phraseology, such as node, mesh, delta, wye, etc., is borrowed from ordinary network theory for simple concepts in switching circuits.

Series-Parallel Two-Terminal Circuits Fundamental Definitions and Postulates

We shall limit our treatment of circuits containing only relay contacts and switches, and therefore at any given time the circuit between any two terminals must be either **open (infinite impedance)** or **closed (zero impedance)**. Let us associate a symbol X_{ab} or more simply X , with the terminals a and b . This variable, a function of time, will be called the hindrance of the two-terminal circuit a - b . The symbol 0 (zero) will be used to represent the hindrance of a closed circuit, and the symbol 1 (unity) to represent the hindrance of an open circuit. Thus when the circuit a - b is open $X_{ab} = 1$ and when closed $X_{ab} = 0$. Two hindrances X_{ab} and X_{cd} will be said to be equal if whenever the circuit a - b is open, the circuit c - d is open, and whenever a - b is closed, c - d is closed. Now let the symbol $+$ (plus) be defined to mean the series connection of the two-terminal circuits whose hindrances are added together. Thus $X_{ab} + X_{cd}$ is the hindrance of the circuit a - d when b and c are connected together. Similarly the product of two hindrances $X_{ab} \cdot X_{cd}$ or more briefly $X_{ab}X_{cd}$ will be defined to mean the hindrance of the circuit formed by connecting the circuits a - b and c - d in parallel.

香农硕士论文中的基本定义与公理

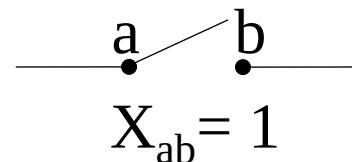
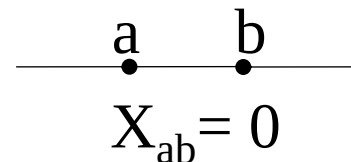
• 假设研究对象为只含有继电器触点和开关的电路。

• 电路只有两个状态：开，关

• X_{ab} ：触点a和b之间的“阻抗”（Hindrane）

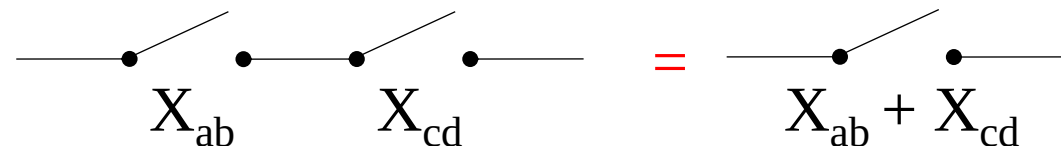
• $X_{ab}=0$ ：电路连通，阻抗为0

• $X_{ab}=1$ ：电路断开，阻抗为无穷大



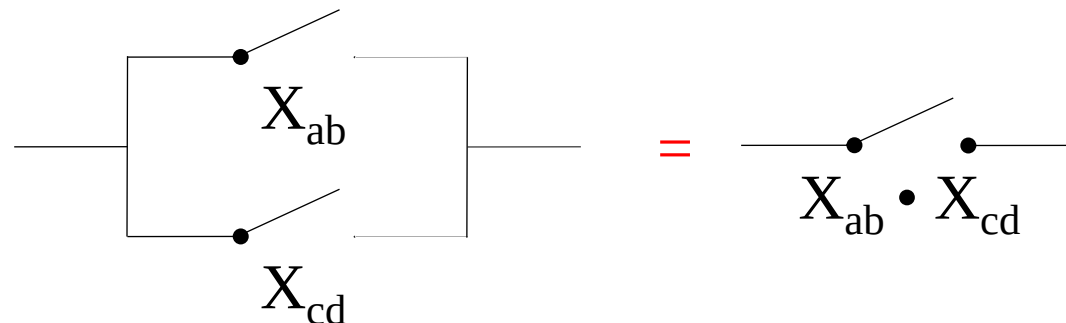
• 串联电路： $X_{ab} + X_{cd}$

• $0+0=0$, $0+1=1$, $1+0=1$, $1+1=1$



• 并联电路： $X_{ab} \cdot X_{cd}$

• $0 \cdot 0=0$, $0 \cdot 1=0$, $1 \cdot 0=0$, $1 \cdot 1=1$



布尔代数基础

- 逻辑运算
- 布尔表达式和真值表
- 逻辑代数定理及规则
- 代数化简法

各种逻辑运算

- 基本逻辑运算 (**Basic Operations**)

- 与 (**AND**)

- 或 (**OR**)

- 非 (**NOT**)

- 复合逻辑运算 (**Other Operations**)

基本运算——AND

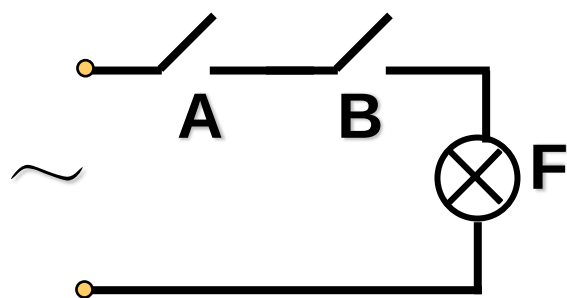
1. AND（逻辑“与” / “乘”）

① $F = A \cdot B$

逻辑赋值：

A/B=1：开关闭合

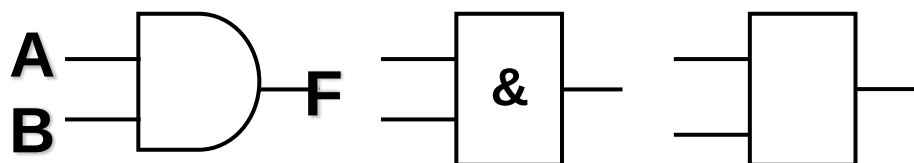
F=1：表示灯亮



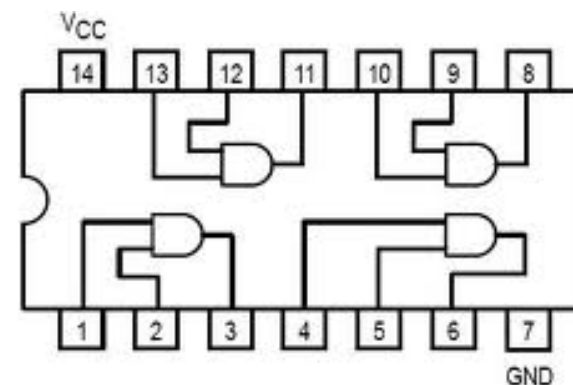
真值表

AB	F
0 0	0
0 1	0
1 0	0
1 1	1

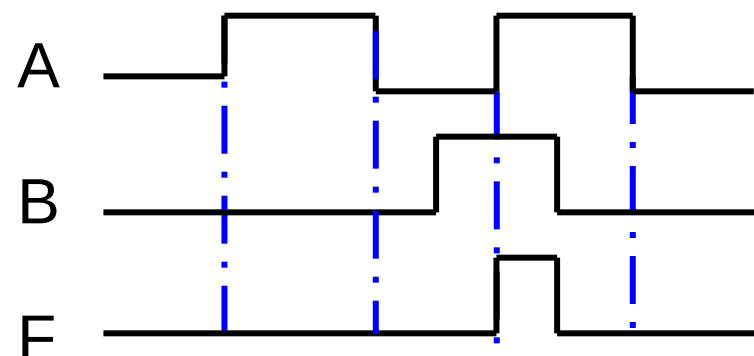
② 逻辑符号



③ 常用芯片：74LS08



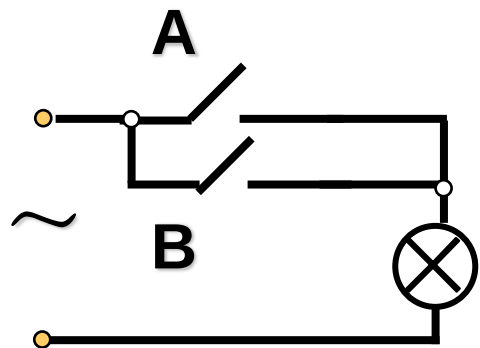
波形图



基本运算——OR

2. OR（逻辑“或” / “加”）

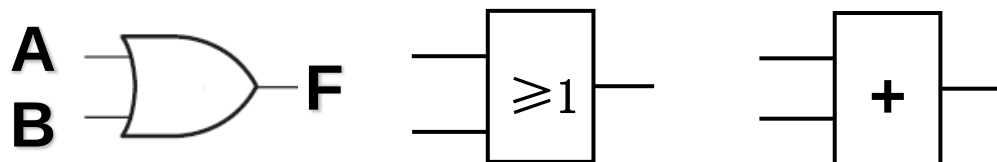
① $F = A + B$



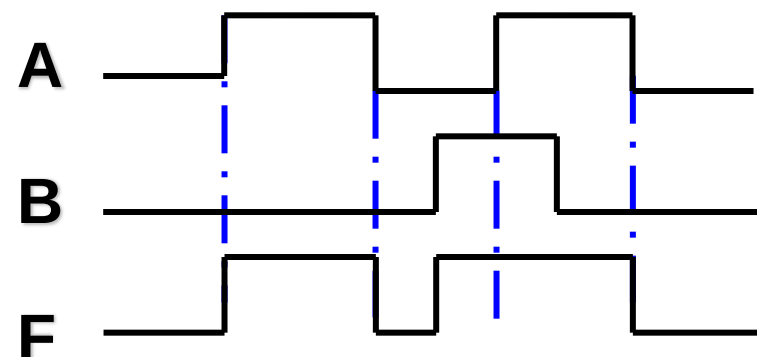
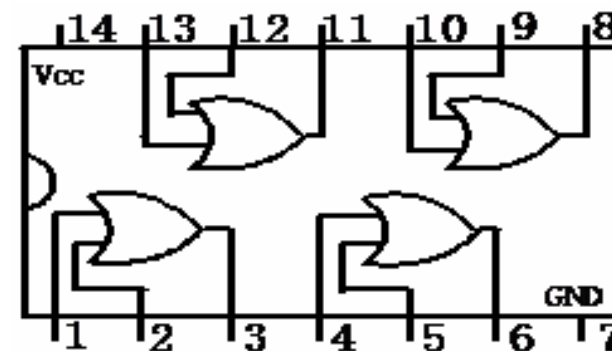
真值表

AB	F
0 0	0
0 1	1
1 0	1
1 1	1

② 逻辑符号



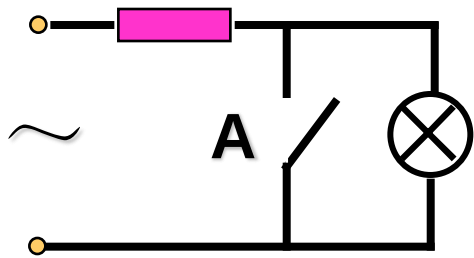
③ 常用芯片：74LS32



基本运算——NOT

3. NOT（逻辑“非”/反相器）

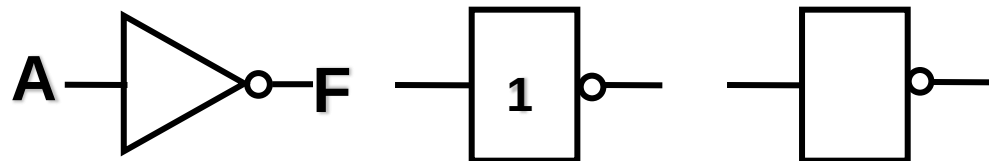
① $F = \bar{A}$ or $F = A'$



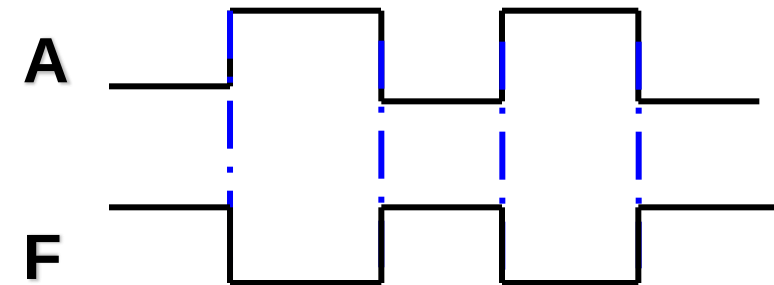
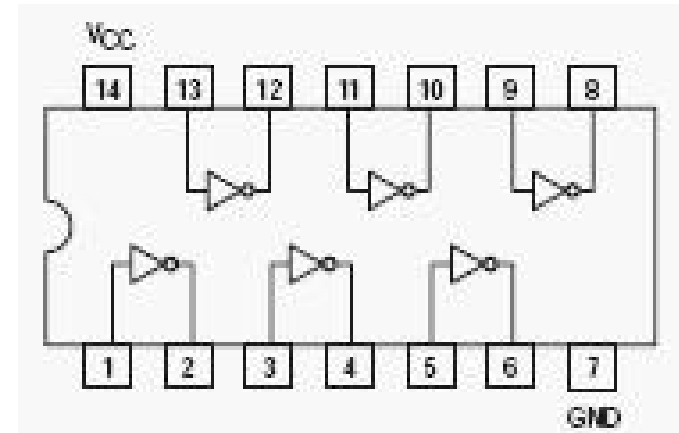
真值表

A	F
0	1
1	0

② 逻辑符号



③ 常用芯片：74LS04



各种逻辑运算

- 基本逻辑运算 (**Basic Operations**)
 - 与 (**AND**)
 - 或 (**OR**)
 - 非 (**NOT**)
- 复合逻辑运算 (**Other Operations**)

复合逻辑运算——NAND

4. 与非

$$F = (A \cdot B)' = \overline{A \cdot B}$$



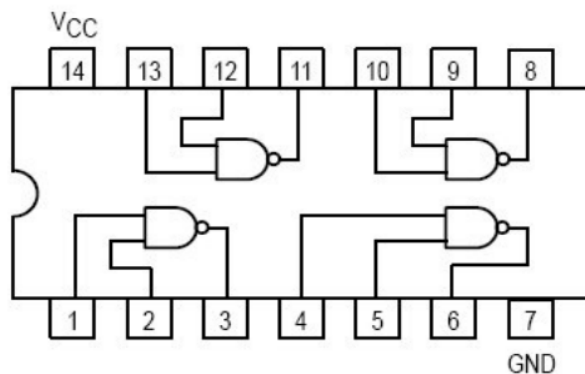
与门+非门

真值表

AB	F
0 0	1
0 1	1
1 0	1
1 1	0

有0出1，全1出0

■ 常用芯片： 74LS00



复合逻辑运算——NOR

5. 或非 (NOR)

$$F = \overline{A+B}$$

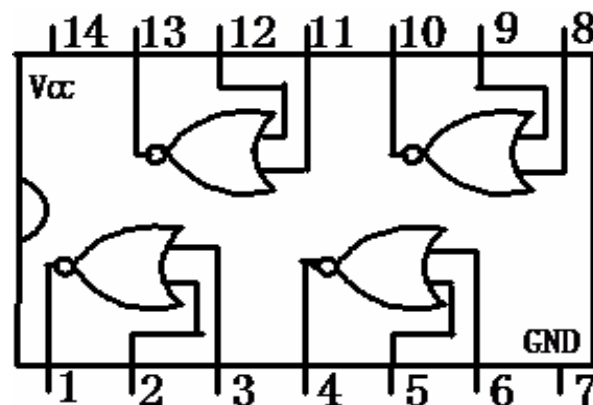


或门+非门

真值表

AB	F
0 0	1
0 1	0
1 0	0
1 1	0

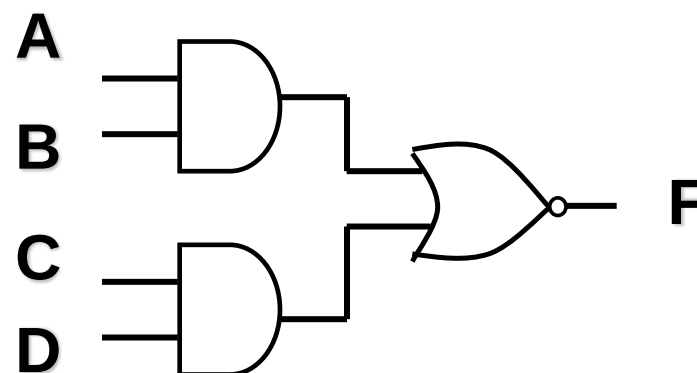
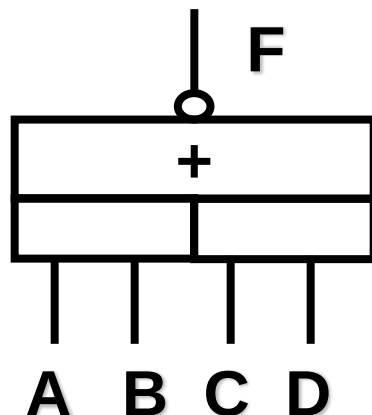
■ 常用芯片: 74LS02



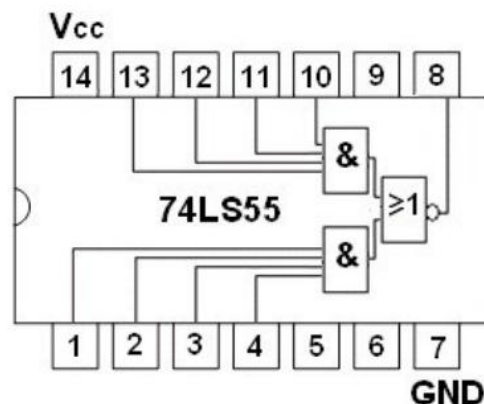
复合逻辑运算——NAND-OR-NOT

6. 与或非 (AND-OR-NOT)

$$F = \overline{AB + CD}$$



■ 常用芯片: 74LS51, 74LS55



复合逻辑运算——异或 (Exclusive-OR)

7. 异或 (Exclusive-OR)

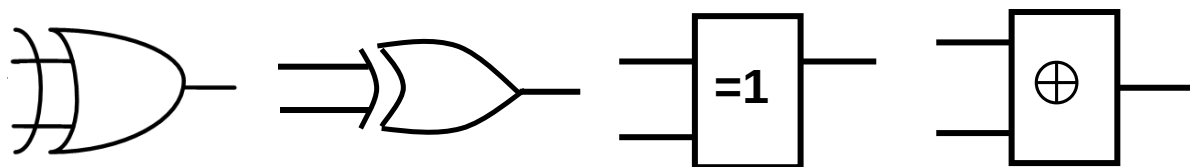
① $F = A \oplus B = \bar{A}B + A\bar{B}$

真值表

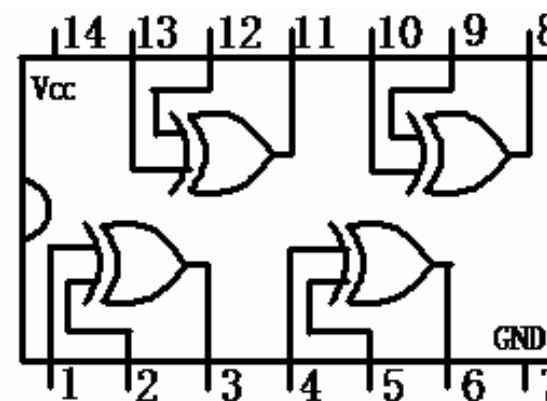
AB	F
0 0	0
0 1	1
1 0	1
1 1	0

不同为1，相同为0

② 逻辑符号



③ 常用芯片: 74LS86



复合逻辑运算——同或 (Exclusive-NOR)

8. 同或 (Exclusive-NOR)

① $F = A \equiv B$ or

$$F = A \odot B = \bar{A}\bar{B} + AB$$

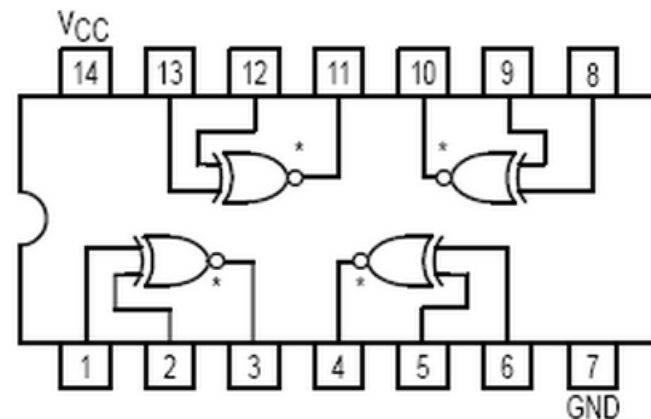
$$A \odot B = (A \oplus B)'$$

真值表

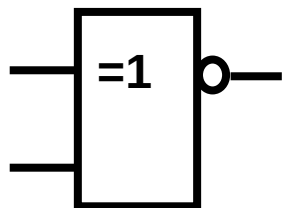
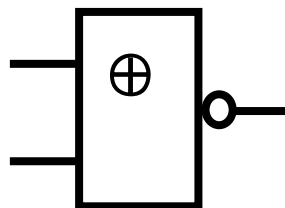
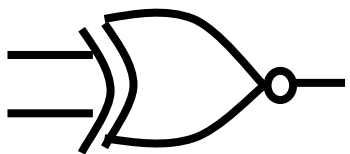
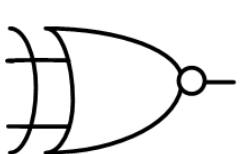
AB	F
0 0	1
0 1	0
1 0	0
1 1	1

相同为1，不同为0

③ 常用芯片: 74LS266



② 逻辑符号



异或、同或的性质

☺异或运算的性质

1. 交换律: $A \oplus B = B \oplus A$

2. 结合律: $A \oplus (B \oplus C) = (B \oplus A) \oplus C$

3. 分配律: $A(B \oplus C) = AB \oplus AC$

4.

$A \oplus 1 = A'$	$A \oplus 0 = A$
$A \oplus A = 0$	$A \oplus A' = 1$



推论:

偶数个“1”异或运算时,
则函数为“0”;

奇数个“1”异或运算时,
则函数为“1”。

异或、同或的性质

☺异或运算的性质

1. 交换律: $A \oplus B = B \oplus A$
2. 结合律: $A \oplus (B \oplus C) = (B \oplus A) \oplus C$
3. 分配律: $A(B \oplus C) = AB \oplus AC$
4.
$$\begin{array}{ll} A \oplus 1 = A' & A \oplus 0 = A \\ A \oplus A = 0 & A \oplus A' = 1 \end{array}$$

☺同或运算的性质

$$A \odot 1 = A$$

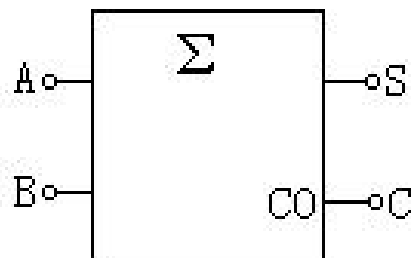
$$A \odot 0 = A'$$

$$A \odot A = 1$$

$$A \odot A' = 0$$

异或门的应用——加法器

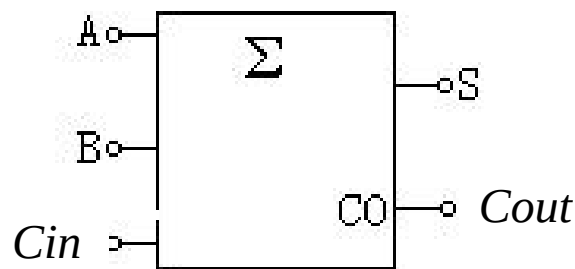
• 半加器 (Half-adder)



半加器逻辑符号

- 半加器只考虑两个1位二进制数相加，不考虑低位的进位。
- A、B是两个加数；
- S是相加的和，C是向高位的进位。

• 全加器 (Full-adder)

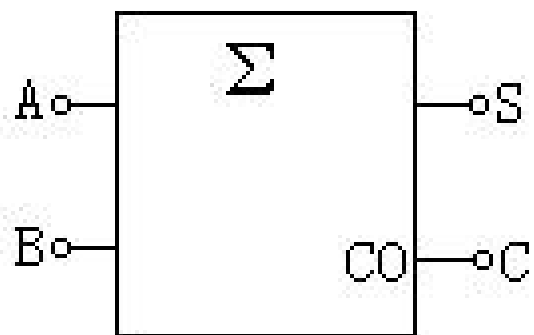


全加器逻辑符号

- 全加器除了加数和被加数外，还要考虑低位的进位。
- A、B是两个加数，Cin是来自低位的进位；
- S是相加的和，Cout是向高位的进位。

异或门的应用——半加器

• 半加器 (Half-adder)



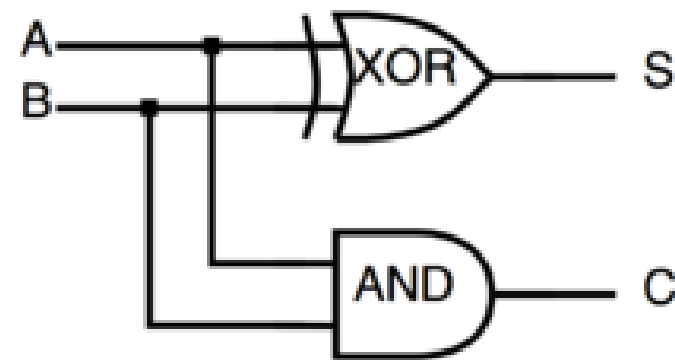
半加器逻辑符号

输入		输出	
A	B	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

逻辑表达式:

$$S = A \oplus B$$

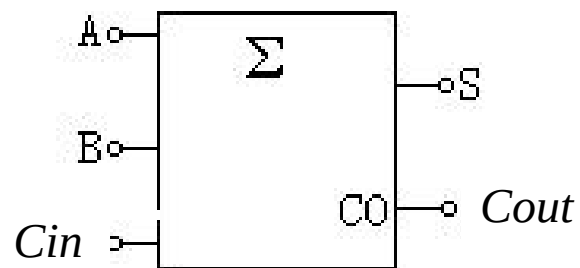
$$C = A \cdot B$$



半加器的逻辑实现

异或门的应用——全加器

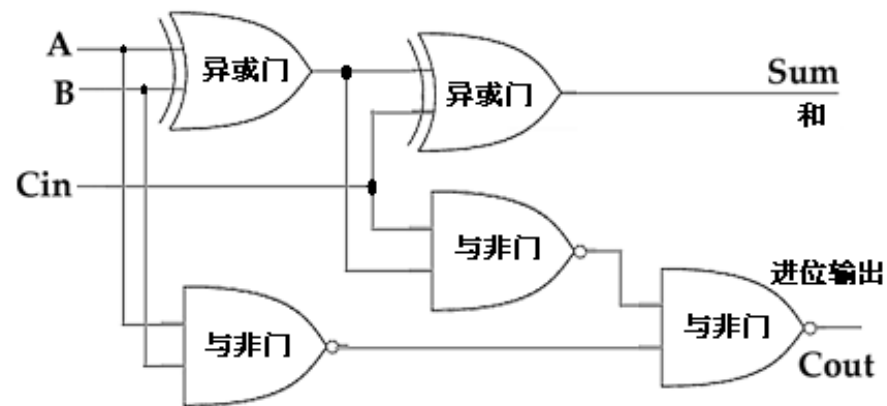
• 全加器 (Full-adder)



全加器逻辑符号

输入			输出	
C_{in}	A	B	C_{out}	Sum
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

$$Sum = A \oplus B \oplus C_{in}$$
$$C_{out} = A \cdot B + C_{in} \cdot (A \oplus B)$$



异或门的应用——等值比较器

如何构造1位等值比较器？

DIFF=0，说明等值

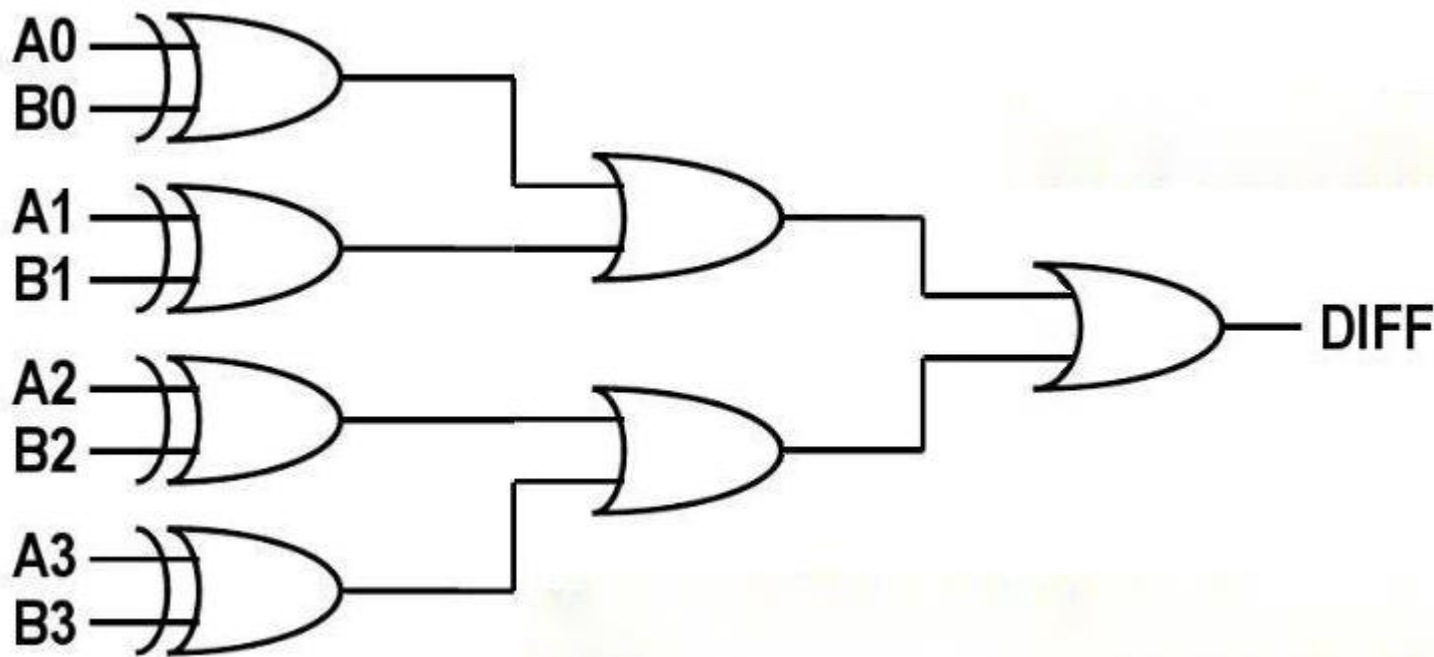


DIFF : different

如何实现4位等值比较器？

$A_3A_2A_1A_0$

$B_3B_2B_1B_0$



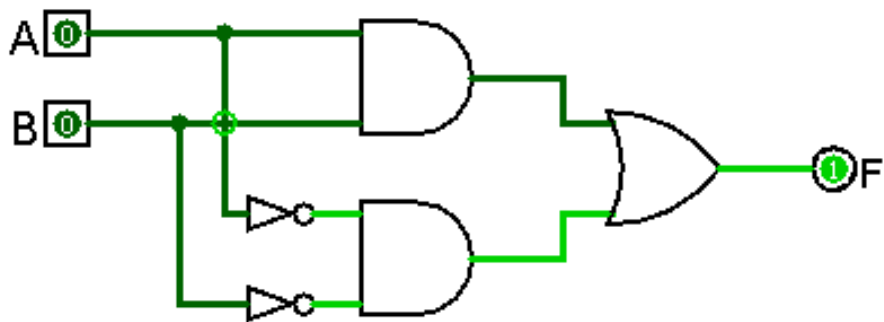
布尔代数基础

- 逻辑运算
- 布尔表达式和真值表
- 逻辑代数定理及规则
- 代数化简法

布尔表达式和真值表

布尔表达式（**Boolean Expressions**）：是由一个或者多个变量和逻辑运算符（与、或、非等）组成的式子。

逻辑图：



$$F = AB + \bar{A}\bar{B}$$

真值表：

AB	F
0 0	1
0 1	0
1 0	0
1 1	1

■ n 个输入变量有 2^n 种取值组合

逻辑表达式如何判定相等？

- 如果两个逻辑表达式的真值表相同，则两个函数式相等

$$AB' + C = (A + C)(B' + C)$$

A	B	C	$AB' + C$	$(A + C)(B' + C)$
0	0	0	0	0
0	0	1	1	1
0	1	0	0	0
0	1	1	1	1
1	0	0	1	1
1	0	1	1	1
1	1	0	0	0
1	1	1	1	1

真值表的
唯一性

布尔代数基础

- 逻辑运算
- 布尔表达式和真值表
- 布尔（逻辑）代数定理及规则
- 代数化简法

布尔代数的公理和定理

公理 (Axiom)

$$(A_1) \mathbf{0 \cdot 0 = 0}$$

$$(A_2) \mathbf{0 \cdot 1 = 1 \cdot 0 = 0}$$

$$(A_3) \mathbf{1 \cdot 1 = 1}$$

$$(A_4) \mathbf{\bar{0} = 1}$$

$$(A_5) \mathbf{\text{If } A \neq 0 \text{ then } A = 1}$$

$$(A_{1D}) \mathbf{0 + 0 = 0}$$

$$(A_{2D}) \mathbf{1 + 0 = 0 + 1 = 1}$$

$$(A_{3D}) \mathbf{1 + 1 = 1}$$

$$(A_{4D}) \mathbf{\bar{1} = 0}$$

$$(A_{5D}) \mathbf{\text{If } A \neq 1 \text{ then } A = 0}$$

单变量定理 (Theorem)

$$(T_1) \quad A + 0 = A$$

$$(T_{1D}) \quad A \cdot 0 = 0$$

$$(T_2) \quad A + 1 = 1$$

$$(T_{2D}) \quad A \cdot 1 = A$$

$$(T_3) \quad A + \bar{A} = 1$$

$$(T_{3D}) \quad A \cdot \bar{A} = 0$$

$$(T_4) \quad A + A = A$$

$$(T_{4D}) \quad A \cdot A = A$$

$$(T_5) \quad \bar{\bar{A}} = A$$

0—1律

符合对偶定理

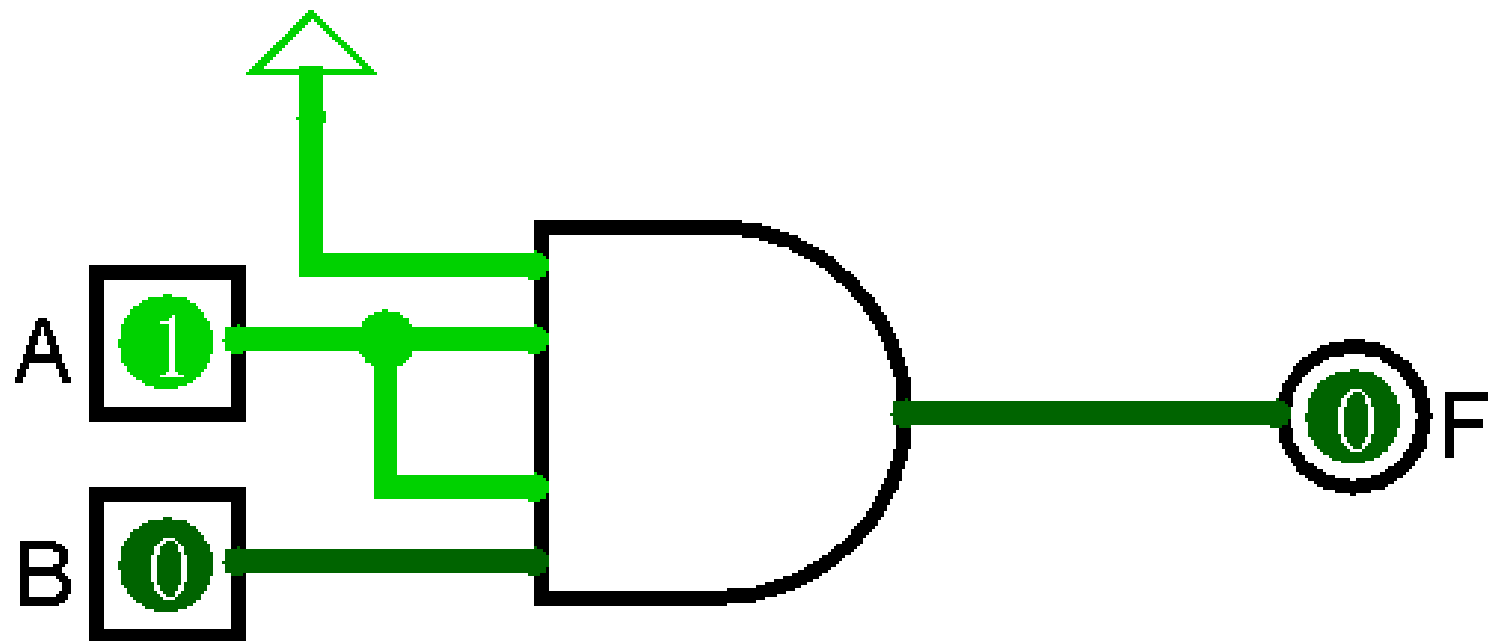
互补律

重叠律

还原律

单变量定理应用

- 多输入逻辑门中不用的输入端怎么办？



二变量定理和三变量定理

交换律

$$(T_6) \mathbf{A+B = B+A}$$

$$(T_{6D}) \mathbf{A \cdot B = B \cdot A}$$

结合律

$$(T_7) \mathbf{(A+B)+C = A+(B+C)}$$

$$(T_{7D}) \mathbf{(A \cdot B) \cdot C = A \cdot (B \cdot C)}$$

分配律

$$(T_8) \mathbf{A \cdot (B+C) = AB+AC}$$

$$(T_{8D}) \mathbf{A+BC = (A+B) \cdot (A+C)}$$

第二分配律

普通代数不支持

二变量定理和三变量定理——续

$$(T_9) \quad A + AB = A$$

$$(T_{10}) \quad AB + A\bar{B} = A$$

$$(T_{11}) \quad A + \bar{A}B = A + B$$

$$A + \bar{A}B$$

$$= A + AB + \bar{A}B$$

$$= A + B$$

$$(T_{9D}) \quad A(A + B) = A \quad (\text{吸收律})$$

$$(T_{10D}) \quad (A + B)(A + \bar{B}) = A \quad (\text{合并律})$$

$$(\text{消除律})$$

摩根定律

$$(X + Y)' = X'Y'$$

$$(XY)' = X' + Y'$$

二变量定理和三变量定理——续

$$(T_9) \quad A + AB = A$$

$$(T_9D) \quad A(A + B) = A \quad (\text{吸收律})$$

$$(T_{10}) \quad AB + A\bar{B} = A$$

$$(T_{10D}) \quad (A+B)(A+\bar{B}) = A \quad (\text{合并律})$$

$$(T_{11}) \quad A + \bar{A}B = A + B$$

(消除律)

$$(T_{12}) \quad AB + \bar{A}C + BC = AB + \bar{A}C$$

(蕴含律)

$$(T_{12D}) \quad (A+B)(A' + C)(B+C) = (A+B)(A' + C)$$

$$= AB + \bar{A}C + (A + \bar{A})BC$$

$$= AB + \bar{A}C + ABC + \bar{A}BC$$

$$= AB + \bar{A}C$$

二变量定理和三变量定理——续

$$(T_9) \quad A + AB = A$$

$$(T_9D) \quad A(A + B) = A \quad (\text{吸收律})$$

$$(T_{10}) \quad AB + A\bar{B} = A$$

$$(T_{10D}) \quad (A+B)(A+\bar{B}) = A \quad (\text{合并律})$$

$$(T_{11}) \quad A + \bar{A}B = A + B$$

(消除律)

$$(T_{12}) \quad AB + \bar{A}C + BC = AB + \bar{A}C$$

(蕴含律)

$$(T_{12D}) \quad (A+B)(B+C)(A'+C) = (A+B)(A'+C)$$

$$(T_{13}) \quad \overline{A\bar{B} + \bar{A}B} = \bar{A}\bar{B} + AB$$

$$\overline{A\bar{B} + \bar{A}B}$$

$$= \overline{A\bar{B}} \cdot \overline{\bar{A}B}$$

$$= (\bar{A} + B) \cdot (A + \bar{B})$$

$$= \bar{A}\bar{B} + AB$$

特殊定理——对偶规则 (Inference of Dual Rule)

变量: 不变

运算符:

$\cdot$	$\longrightarrow$	$+$
$+$	$\longrightarrow$	$\cdot$
$\oplus$	$\longrightarrow$	$\odot$
$\odot$	$\longrightarrow$	$\oplus$

不能改变原来的优先级

常量: $0 \longrightarrow 1$ $1 \longrightarrow 0$

- 求对偶式的另一种方法
 - 对整个表达式取反，然后再对每个变量取反。

对偶规则运用实例

$$F=A \cdot (B+C) \xrightarrow{\text{对偶}} (F)^D = A+B \cdot C$$

$$F' = (A \cdot (B+C))' = A' + (B+C)' = A' + B'C' \quad (\text{对整个表达式取反})$$

$$F^D = A+BC \quad (\text{对每个变量取反})$$

$$F=A \cdot \bar{B} + AC \xrightarrow{\text{对偶}} (F)^D = (A+\bar{B}) \cdot (A+C)$$

$$F=\overline{\bar{A} \cdot \bar{B} \cdot \bar{C}} \xrightarrow{\text{对偶}} (F)^D = \overline{\bar{A} + \bar{B} + \bar{C}}$$

对偶规则的性质

① $F \xleftrightarrow{\text{Dual Rule}} (F)^D$

② 两个逻辑表达式相等，它们的对偶式也相等

$$A + BCD = (A + B)(A + C)(A + D)$$



Dual Rule



Dual Rule

$$A \cdot (B + C + D) = AB + AC + AD$$

对偶规则的性质

例1.1.5 试利用对偶规则证明分配律式子 $A + BC = (A+B)(A+C)$ 成立。

证明：设 $Y = A + BC$ ， $G = (A+B)(A+C)$ ，

则它们的对偶式为

$$\begin{aligned} Y^D &= A \cdot (B + C) = AB + AC \\ G^D &= AB + AC \end{aligned}$$

由于 $Y^D = G^D$

故 $Y = G$ ，即 $A + BC = (A+B)(A+C)$ 。

对偶规则的性质

例1.1.6 试利用对偶规则证明吸收律 $A+A'B=A+B$ 成立。

证明：设 $Y = A + A'B$ $G = A + B$

则它们的对偶式为

$$Y^D = A \cdot (A' + B) = A \cdot A' + AB = AB \quad G^D = AB$$

由于 $Y^D = G^D$

故 $Y=G$ ，即 $A + A'B = A + B$

反演定理

- 对任一逻辑表达式 Y , 求 Y'

$$\bullet \longrightarrow +$$

$$+ \longrightarrow \bullet$$

$$0 \longrightarrow 1$$

$$1 \longrightarrow 0$$

$$\text{原变量} \longrightarrow \text{反变量}$$

$$\text{反变量} \longrightarrow \text{原变量}$$

- De. Morgan定理是反演定理的一个**特例**。

反演定理

例2.4.3 已知 $Y = A(B + C) + C'D$ ，求 Y' 。

1. 由反演定理直接求反；
2. 由摩根定理推导。

反演定理

例2.4.3 已知 $Y=A(B+C)+C'D$ ，求 Y' 。

解：由反演定理直接求反

$$\begin{aligned} Y' &= (A' + B'C')(C + D') \\ &= A'C + A'D' + B'C'C + B'C'D' \\ &= A'C + A'D' + B'C'D' \end{aligned}$$

先与、后或

注意：

1. 保持先括号、然后与、最后或的顺序；
2. 保留括号；
3. 不属于单个变量上的“反号”要保留不变。

反演定理

例2.4.3 已知 $Y = A(B + C) + C'D$ ，求 Y' 。

解：由反演定理直接求反

$$\begin{aligned} Y' &= (A' + B'C')(C + D') \\ &= A'C + A'D' + B'C'C + B'C'D' \\ &= A'C + A'D' + B'C'D' \end{aligned}$$

由摩根定理推导得

$$\begin{aligned} Y' &= [A(B + C) + C'D]' = [A(B + C)]' \cdot (C'D)' \\ &= [A' + (B + C)'] \cdot (C + D') \\ &= (A' + B'C') \cdot (C + D') \\ &= A'C + A'D' + B'C'C + B'C'D' \\ &= A'C + A'D' + B'C'D' \end{aligned}$$

n 变量定理

$$(T_{14}) X + X + \dots + X = X$$

$$(T_{14D}) X \bullet X \bullet \dots \bullet X = X \quad (\text{广义同一律})$$

$$(T_{15}) (X_1 \bullet X_2 \bullet \dots \bullet X_n)' = X_1' + X_2' + \dots + X_n'$$

$$(T_{15}) (X_1 + X_2 + \dots + X_n)' = X_1' \bullet X_2' \bullet \dots \bullet X_n' \quad (\text{德·摩根定理})$$

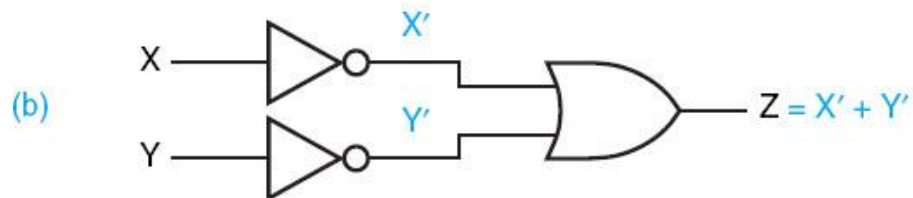
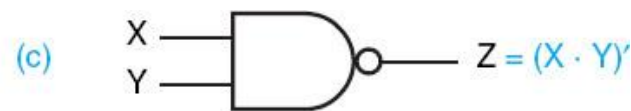
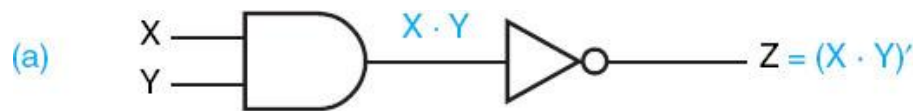
$$(T_{16}) F(X_1, X_2, \dots, X_n) = X_1 \bullet F(1, X_2, \dots, X_n) + X_1' \bullet F(0, X_2, \dots, X_n)$$

$$(T_{16D}) F(X_1, X_2, \dots, X_n) = [X_1 + F(0, X_2, \dots, X_n)] \bullet [X_1' + F(1, X_2, \dots, X_n)]$$

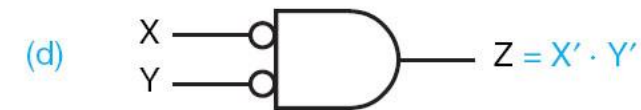
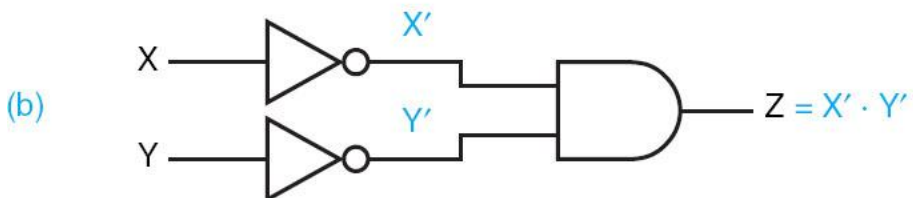
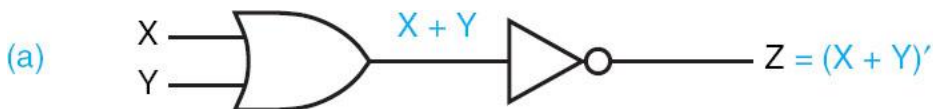
(香农展开定理)

根据德·摩根定理的等效电路

$$(T15) \quad (X_1 \cdot X_2 \cdot \dots \cdot X_n)' = X_1' + X_2' + \dots + X_n'$$



$$(T15D) \quad (X_1 + X_2 + \dots + X_n)' = X_1' \cdot X_2' \cdot \dots \cdot X_n'$$



布尔代数基础

- 逻辑运算
- 布尔表达式和真值表
- 逻辑代数定理及规则
- 代数化简法

代数化简

一个逻辑函数有多种不同的表达式

$$F=AB+A\bar{C} \quad \dots\dots \text{与-或}$$

$$\overline{\overline{AB+A\bar{C}}}$$

$$=\overline{\overline{AB}} \cdot \overline{\overline{A\bar{C}}} \quad \dots\dots \text{与非-与非}$$

$$=(\overline{A+B}) \cdot (\overline{A+C}) \quad \dots\dots \text{或-与非}$$

$$=(\overline{A+B}) + (\overline{A+C}) \quad \dots\dots \text{或非-或}$$

与或式——积之和

$$F=(A+B) \cdot (A+\bar{C}) \quad \dots\dots \text{或-与}$$

$$=\overline{\overline{(A+B) \cdot (A+\bar{C})}}$$

$$=\overline{(\overline{A+B}) + (\overline{A+\bar{C}})} \quad \dots\dots \text{或非-或非}$$

$$=\overline{\overline{A} \cdot \overline{B}} + \overline{\overline{A} \cdot C} \quad \dots\dots \text{与-或非}$$

$$=\overline{\overline{A} \cdot \overline{B}} \cdot \overline{\overline{A} \cdot C} \quad \dots\dots \text{与非-与}$$

或与式——和之积

同一类型的表达式也**不是**唯一的

$$F=AB+\bar{A}C$$

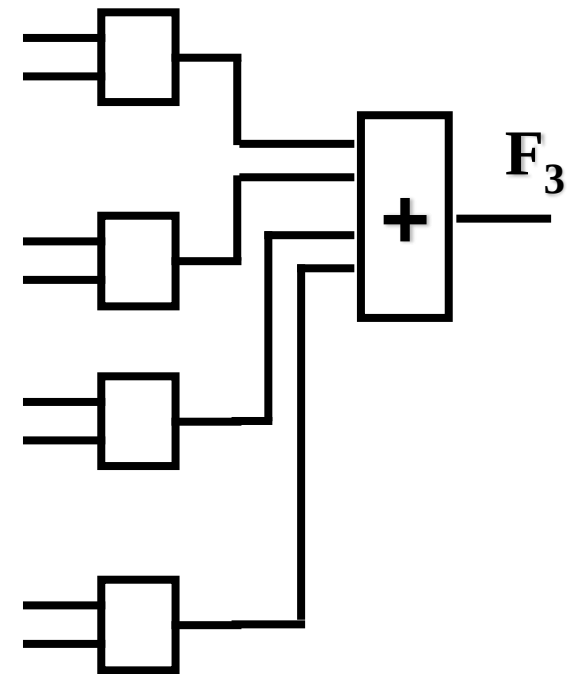
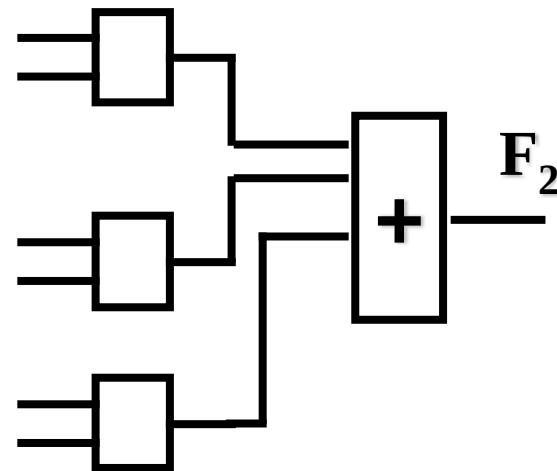
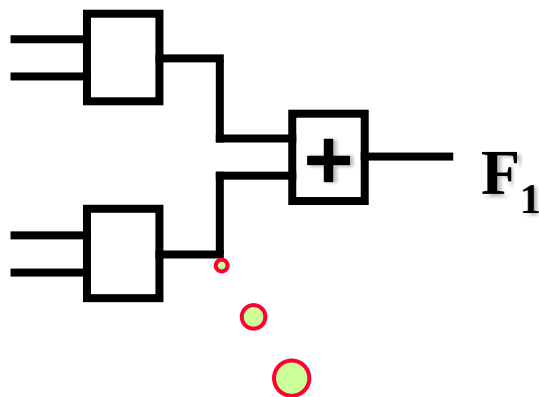
.....① F_1

$$=AB+\bar{A}C+BC$$

.....② F_2

$$=ABC+AB\bar{C}+\bar{A}BC+\bar{A}\bar{B}C$$

.....③ F_3



最简，元件少，可靠

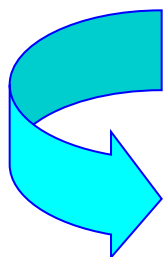
代数化简

最简 (Minimum Expressions) ?

① 与项 (和项) 的个数最少

② 每个与项 (和项) 中变量的个数最少

最简与或
最简或与



成本最低

① 逻辑门的数量最少

② 逻辑门的输入个数最少

目的:

- 降低成本
- 提高可靠性

方法: {
■ 代数法 (Algebraic techniques)
■ 卡诺图法 (K. map method)

代数化简法例子—1

$$F = A + \underbrace{A\bar{B}\bar{C}} + \bar{A}CD + \bar{C}E + \bar{D}E$$

$$= A + \underbrace{\bar{A}CD} + \bar{C}E + \bar{D}E$$

$$= A + CD + \underbrace{\bar{C}E + \bar{D}E}$$

$$= A + CD + E(\bar{C} + \bar{D})$$

$$= A + CD + \underbrace{E\bar{C}\bar{D}}$$

$$= A + CD + E$$

吸收律

消除律

分配律

摩根律

消除律

代数化简法例2

$$F = \underline{AB} + A\bar{C} + \bar{B}C + B\bar{C} + \bar{B}D + B\bar{D} + ADE(F+G)$$

$$= A(\underline{\bar{B}C}) + \bar{B}C + B\bar{C} + \bar{B}D + B\bar{D} + ADE(F+G)$$

$$= \underline{A} + \bar{B}C + B\bar{C} + \bar{B}D + B\bar{D} + \underline{ADE(F+G)}$$

$$= A + \underline{\bar{B}C + B\bar{C} + \bar{B}D + B\bar{D}} + C\bar{D}$$

$$= A + \bar{B}C + B\bar{C} + \bar{B}D + B\bar{D} + C\bar{D}$$

$$= A + \bar{B}C + B\bar{C} + \bar{B}D + C\bar{D}$$

$$= A + B\bar{C} + \bar{B}D + C\bar{D}$$

代数化简法例子—3,4

$$F = (\bar{B}+D)(\bar{B}+D+A+G)(C+E)(\bar{C}+G)(A+E+G)$$

对偶规则

$$F^D = \bar{B}D + \bar{B}DAG + CE + \bar{C}G + AEG$$

$$= \bar{B}D + \underbrace{CE + \bar{C}G}_{\text{blue bracket}} + \cancel{AEG}$$

$$= \bar{B}D + CE + \bar{C}G$$

对偶规则

$$F = (\bar{B}+D)(C+E)(\bar{C}+G)$$

$$= \dots$$

代数化简法优缺点

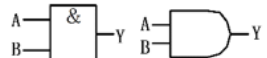
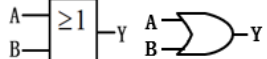
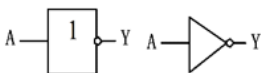
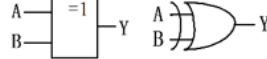
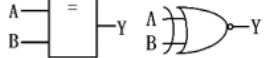
- 优点：
 - 不受变量数目的约束
 - 对公理、定理和规则十分熟练时，化简较方便
- 缺点：
 - 技巧性强
 - 在很多情况下难以判断化简结果是否最简

小 结

- 各种逻辑运算
- 布尔表达式和真值表
- 逻辑代数定理及规则
- 代数化简法

上节课回顾

基本逻辑运算

名称	逻辑函数式	门电路
与 (AND)	$Y = A \cdot B$	
或 (OR)	$Y = A + B$	
非 (NOT)	$Y = A'$	
异或	$Y = A \oplus B = AB' + A'B$	
同或	$Y = A \odot B = (A \oplus B)' = AB + A'B'$	

基本公式

摩根定理: $(AB)' = A' + B'$ $(A + B)' = A' \cdot B'$

反演定理

对偶定理